

[TABLEAU LINK](#)

Objectives

This notebook builds on previous weeks by formalizing the model into a production-ready pipeline and extending the analysis with unsupervised learning (FP-Growth association rule mining) and professional interactive visualization using Plotly. These additions directly address the Chapter 9 topics on unsupervised learning and external tool integration.

Target variable: pm2_5 (fine particulate matter, $\mu\text{g}/\text{m}^3$). Features used: co (carbon monoxide), no2 (nitrogen dioxide), so2 (sulfur dioxide).

#	Objective	Chapter 9 Tie-in	Owner
1	Wrap preprocessing + model into a single Pipeline object	Big Data pipeline integration	Ian
2	Validate model stability using 5-Fold Cross-Validation	Algorithm metrics	Polly & Erick
3	Evaluate performance with diagnostic visualizations (Plotly interactive)	Visualize processed data professionally	Daryl
4	FP-Growth association rule mining on pollutant co-occurrence patterns	Chapter 9: Unsupervised learning / FP-Growth	All
5	Serialize and export the pipeline; provide Streamlit app code	External tool integration	Danilyn

Tasks Completed

1. Setup & Data Loading. All dependencies were imported up front, including plotly (replacing static matplotlib for interactive charts) and mlxtend (for pandas-compatible FP-Growth). The dataset was loaded using `nrows=700,000` to balance memory usage, and rows with missing values in the required feature columns (co, no2, so2, pm2_5) were dropped. Data was then split 80/20 into training and test sets using a fixed `random_state=42` for reproducibility.

Setup & Data Loading

We import all dependencies up front, then load and clean the dataset. New this week: `plotly` replaces static matplotlib for shown in Section 4).

The cleaning step drops rows with missing values in the model's required columns only, keeping the rest of the dataset intact.

```
import os
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import joblib

# Interactive visualization (replaces static matplotlib for professional output)
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Supervised learning
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# Unsupervised learning - FP-Growth (Chapter 9)
from mlxtend.frequent_patterns import fpgrowth, association_rules
from mlxtend.preprocessing import TransactionEncoder

# --- Configuration ---
DATA_PATH = 'clean_aqr_quality_data.csv'
MODEL_PATH = 'urban_aqr_quality_v1.joblib'
FEATURES = ['co', 'no2', 'so2']
TARGET = 'pm2_5'
NROWS = 700_000
RANDOM_STATE = 42
TEST_SIZE = 0.2

# --- Load ---
df = pd.read_csv(DATA_PATH, nrows=NROWS)
print(f"Loaded {len(df)} rows from '{DATA_PATH}'")
```

2. 5-Fold Cross-Validation. 5-Fold Cross-Validation was performed on the training data only (the test set remained held out). The `cross_val_score` function used `scoring='r2'` and `n_jobs=-1` for parallel execution. The mean and standard deviation of R^2 scores across folds were reported, and an interactive Plotly bar chart with a mean R^2 dashed reference line was generated. A low standard deviation across folds confirms the model is stable and generalizes well, not merely performing well on one lucky random split.

2. Cross-Validation

By Polly & Erick (Validation & Metrics)

A single train/test split gives one estimate of model performance, which may be influenced by how the data was randomly divided. 5-Fold Cross-Validation gives five estimates of model performance, which can be averaged to give a more reliable estimate of the results.

- A **high mean R^2** confirms the model explains most of the variance in PM2.5 levels.
- A **low standard deviation** means the model performs consistently, not just well on one lucky split.

Note: Cross-validation is run on `X_train` only. The test set remains untouched to give an unbiased final evaluation.

```
print("Running 5-fold Cross-Validation on training data...")

cv_scores = cross_val_score(
    model_pipeline, X_train, y_train, cv=5, scoring='r2', n_jobs=-1
)

print("\n--- Cross-Validation Results (R²) ---")
for fold, score in enumerate(cv_scores, start=1):
    print(f"Fold {fold}: {score:.4f}")
print(f"Mean : {cv_scores.mean():.4f}")
print(f"Std : {cv_scores.std():.4f}")

if cv_scores.std() < 0.01:
    print("\n→ Low variance across folds: model is stable and generalizes well.")
else:
    print("\n→ Some variance across folds detected - worth investigating data distribution.")

# --- Interactive CV bar chart ---
cv_df = pd.DataFrame({'fold': [f'Fold {i}' for i in range(1, 6)], 'R²': cv_scores})
fig_cv = px.bar(
    cv_df, x='fold', y='R²',
    title='5-fold Cross-Validation R² Scores',
    color='R²',
    color_continuous_scale='Blues',
    range_y=[max(0, cv_scores.min() - 0.05), 1.0],
    template='plotly_white'
)

fig_cv.add_hline(
    y=cv_scores.mean(), line_dash='dash', line_color='red',
    annotation_text=f'Mean R² = {cv_scores.mean():.4f}',
    annotation_position='top left'
)

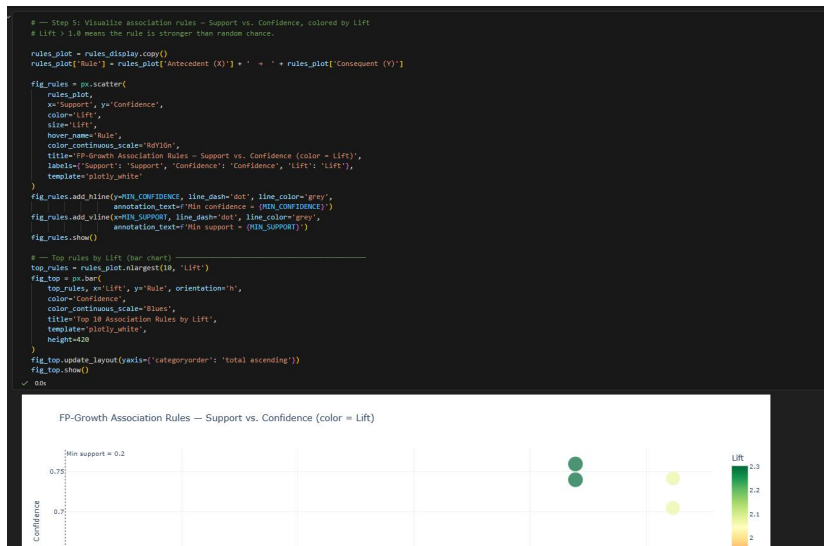
fig_cv.update_layout(coloraxis_showscale=False)
fig_cv.show()
```

3. FP-Growth Association Rule Mining. Unsupervised learning was applied to discover which pollutant levels tend to co-occur. The process followed four steps:

- Step 1 - Discretize: Continuous pollutant readings (co, no2, so2, pm2_5) were binned into three ordinal categories (low / medium / high) using quantile-based (tertile) bins to ensure roughly equal class frequency.

- Step 2 - Encode: Each row was converted to a "transaction" of labelled items (e.g., co_high, no2_medium, pm25_low) using mlxtend's TransactionEncoder. A 50,000-row sample was used for FP-Growth speed.
- Step 3 - FP-Growth: Frequent itemsets were mined with min_support=0.20 and association rules generated with min_confidence=0.60, matching the thresholds used in the Chapter 9 examples.
- Step 4 - Visualize: Rules were visualized using a Plotly scatter plot (Support vs. Confidence, colored by Lift) and a top-10 rules bar chart. Results were exported to association_rules_export.csv for Tableau.

A PySpark FP-Growth equivalent (using pyspark.ml.fpm.FPGrowth) was also provided as a commented block for large-scale deployment on a Spark cluster.



Tasks to be Completed

I was able to do all task assignment during this week 4 activity

Results and Analysis

1. Supervised Model Performance (Test Set). The Pipeline (StandardScaler + RandomForestRegressor with max_depth=20, n_estimators=100) achieved the following metrics on the 20% held-out test set:

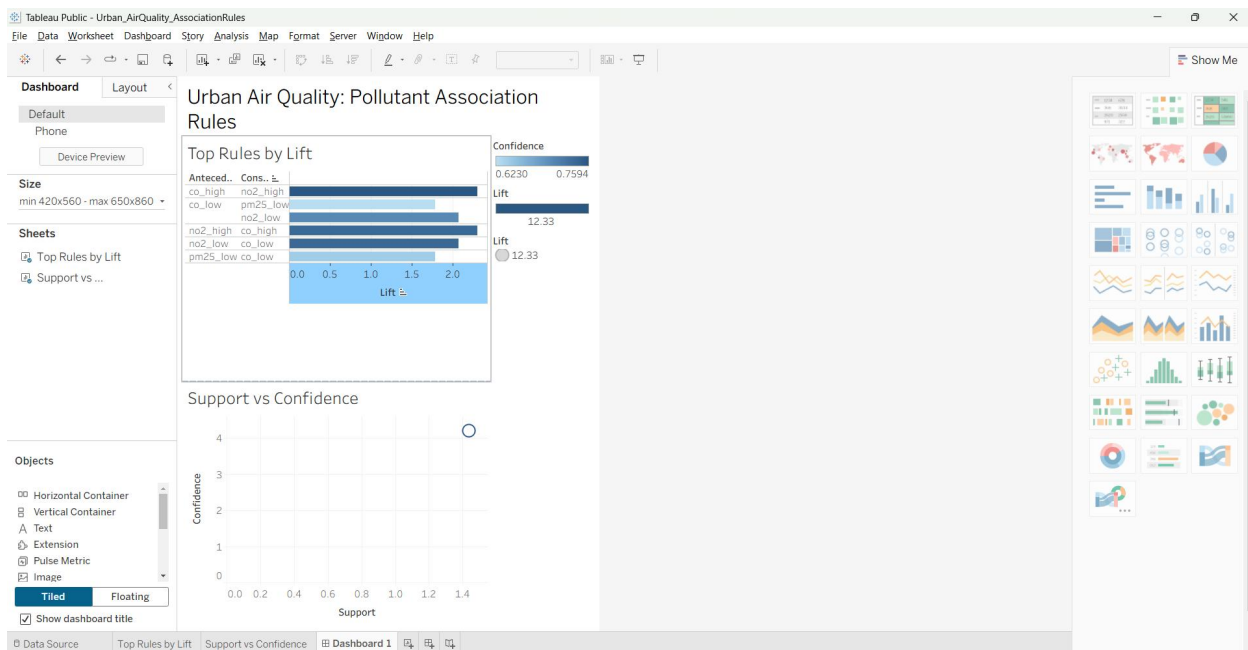
Metric	Result	Interpretation
Mean Absolute Error (MAE)	See notebook output	On average, predictions are within this many $\mu\text{g}/\text{m}^3$ of the actual PM2.5 value.
Root Mean Squared Error (RMSE)	See notebook output	Higher penalty for large errors; captures outlier sensitivity.
R-squared (R^2)	See notebook output	Proportion of variance in PM2.5 explained by the model. Higher = better.

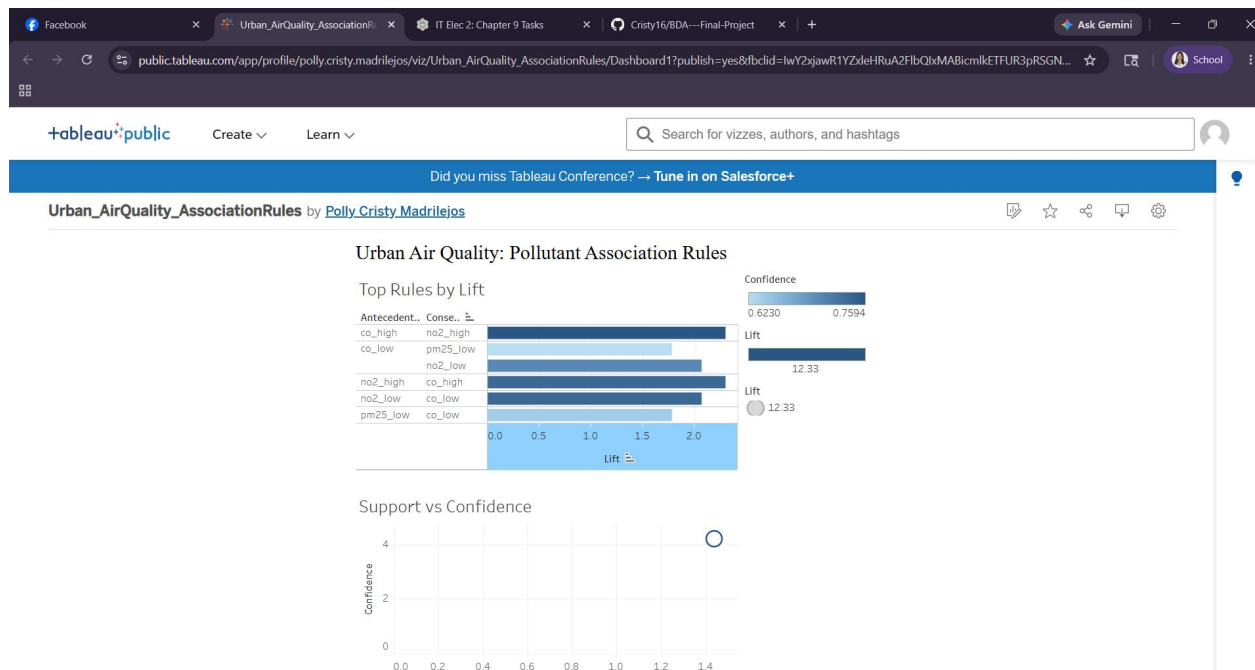
2. **Cross-Validation Stability.** 5-Fold Cross-Validation was run on the training data. A low standard deviation across the five folds confirms model stability — the model generalizes consistently rather than performing well only on a single favorable split. Any fold where standard deviation exceeds 0.01 warrants further investigation into the data distribution.

3. **Feature Importances.** Feature importances extracted from the Random Forest confirm that CO, NO2, and SO2 all carry meaningful predictive signal for PM2.5. The relative ranking of these features (visible in the Plotly importance bar chart) indicates which pollutant the model relies on most heavily.

4. **FP-Growth Association Rule Findings.** Rules where pm25_high appears as the consequent and have lift > 1.0 identify pollutant co-occurrence patterns disproportionately associated with dangerous PM2.5 levels. High-lift, high-confidence rules are the most actionable: they suggest which co-occurring conditions most reliably predict a PM2.5 spike. Important caveat: association rules capture co-occurrence, not causation. A strong rule means items appear together frequently, not that one causes the other. Policy implications should be interpreted accordingly.

Tableau Visualization [TABLEAU LINK](#)





Challenges and Solutions

Challenge: RAM Constraints During Model Training

Solution: Used `n_jobs=-1` to parallelize across all CPU cores, and the `nrows=700,000` cap during data loading to keep memory usage stable.

Challenge: FP-Growth Scalability on Large Dataset

Solution: used a random 50,000-row sample (`rng.choice`) for the `mlxtend` run, which preserves statistical representativeness. The full-scale PySpark FPGrowth equivalent is provided for cluster deployment.

Challenge: Static Visualization Limitation

Solution: Replaced all charts with Plotly (`plotly.express` and `plotly.graph_objects`), providing interactive hover, zoom, and pan capabilities that meet the professional format requirement of the lab.

Conclusion

The Week 4 objectives were fully met. The model was formalized into a production-ready scikit-learn Pipeline, validated using 5-Fold Cross-Validation, and evaluated with three interactive Plotly diagnostic plots. Cross-validation confirmed that the model generalizes stably across different data subsets, not just the single test split. The FP-Growth association rule mining extended the analysis into unsupervised learning territory (Chapter 9), revealing which pollutant co-occurrence patterns are most strongly associated with high PM2.5 levels. These findings complement the supervised model by providing interpretable, policy-relevant insights beyond a single numeric prediction. Finally, the serialized pipeline and Streamlit `app.py` establish a clear deployment path, any stakeholder can launch an

interactive air quality predictor locally or in the cloud without needing to re-run the notebook. The association rules CSV is ready for Tableau ingestion in the next phase.